

Laravel Blog App

A Complete Beginner's Guide to How Your Code Works

Understanding Laravel 13 • MVC • CRUD • Blade Templates

1. What is Laravel?

Laravel is a PHP framework — think of it as a toolkit that gives you a clean, organized way to build websites and web apps. Instead of writing everything from scratch, Laravel provides ready-made tools for things like connecting to a database, handling web requests, and displaying pages.

Your blog app is a great first project because it covers all the core things Laravel does. Before diving into your specific code, it helps to understand the big idea behind how Laravel thinks.

The MVC Pattern

Laravel follows a design pattern called MVC — Model, View, Controller. This is the backbone of how everything is organized. Almost every Laravel project you will ever see uses this pattern.

Model — Handles data. It talks to the database and represents your data as objects in PHP.

View — Handles what the user sees. It is the HTML that gets shown in the browser.

Controller — The middleman. It receives the request from the browser, asks the Model for data, and passes it to the View.

Think of it like a restaurant: The Controller is the waiter (takes your order), the Model is the kitchen (prepares the data), and the View is the plate of food (what you actually see).

2. The Laravel File Structure

When you first open a Laravel project in VS Code it looks overwhelming. Here is a breakdown of the folders that matter for your blog app — and which ones you can mostly ignore for now.

Folders You Use

app/Models/ — Your Post.php model lives here. This is where you define what data your app works with.

app/Http/Controllers/ — Your PostController.php lives here. This is where all your CRUD logic is written.

resources/views/ — All your Blade HTML files live here. This is what the user actually sees.

routes/web.php — This file defines all the URLs your app responds to.

database/migrations/ — These files define your database tables. You ran php artisan migrate to create the posts table.

public/ — The only folder the browser can access directly. Your CSS files go in public/css/.

.env — Your environment config file. This is where you set your database name, username, and password.

Folders You Can Ignore For Now

- **bootstrap/** — Laravel's internal startup code
- **config/** — Advanced configuration files
- **storage/** — Logs, caches, file uploads
- **tests/** — Automated tests
- **vendor/** — All the third-party packages Laravel uses internally (never edit this)
- **node_modules/** — JavaScript dependencies (only used if you run npm)

Rule of thumb: For your blog app, you only really need to care about app/, resources/views/, routes/web.php, database/migrations/, and public/css/.

3. The Database & Migration

Your blog posts need to be stored somewhere. Laravel uses MySQL (or other databases) to store data. You connected Laravel to MySQL using your `.env` file.

The `.env` File

The `.env` file tells Laravel how to connect to your database. The important lines you edited were:

```
DB_DATABASE=lastname_blog_db
DB_USERNAME=root
DB_PASSWORD=yourpassword
```

Laravel reads this file on every request and uses those details to connect to MySQL. You never put your password directly in your PHP code — `.env` keeps it separate and secure.

What is a Migration?

A migration is a PHP file that describes what your database table should look like. Instead of manually creating a table in phpMyAdmin, you write a migration and Laravel creates it for you.

When you ran the command below, Laravel executed your migration and created the posts table in MySQL:

```
php artisan migrate
```

Your migration file created a posts table with these columns:

- `id` — A unique auto-incrementing number for each post
- `title` — A short text field for the post title
- `body` — A longer text field for the post content
- `created_at` — Automatically records when the post was created
- `updated_at` — Automatically records when the post was last updated

Why migrations? Migrations let you version-control your database structure, just like you version-control your code with Git. If you need to change the table later, you create a new migration instead of manually editing the database.

4. The Model — Post.php

The Model is how Laravel represents a row in your database as a PHP object. Your Post model is located at:

```
app/Models/Post.php
```

The key thing you added to this file was:

```
protected $fillable = ['title', 'body'];
```

What does \$fillable do?

By default Laravel protects your model from mass assignment — meaning it won't let you save data from a form all at once unless you explicitly say which fields are allowed. This is a security feature.

By adding title and body to \$fillable, you are telling Laravel: these two fields are safe to fill from form data. Without this line, when you try to create a post, Laravel would throw a `MassAssignmentException` error.

The model also gives you powerful built-in methods for free, without writing any SQL:

- `Post::all()` — Get every post from the database
- `Post::latest()->get()` — Get all posts, newest first
- `Post::create($data)` — Insert a new row
- `$post->update($data)` — Update an existing row
- `$post->delete()` — Delete a row

Laravel Magic: You never wrote a single SQL query like `SELECT * FROM posts`. Laravel's Eloquent ORM (Object Relational Mapper) converts PHP method calls into SQL queries behind the scenes.

5. Routes — web.php

Routes define what happens when someone visits a URL in your app. Your routes file is at:

```
routes/web.php
```

You only needed one line to define all seven routes for your blog:

```
Route::resource('posts', PostController::class);
```

What does Route::resource do?

This one line automatically creates seven different routes for you. It follows a standard convention called RESTful routing. Here is what each route does:

GET /posts — Shows the list of all posts (calls index method)

GET /posts/create — Shows the form to create a new post (calls create method)

POST /posts — Saves a new post to the database (calls store method)

GET /posts/{id} — Shows a single post (calls show method)

GET /posts/{id}/edit — Shows the edit form (calls edit method)

PUT /posts/{id} — Updates an existing post (calls update method)

DELETE /posts/{id} — Deletes a post (calls destroy method)

Why PUT and DELETE? HTML forms only support GET and POST. Laravel uses a hidden field called @method('PUT') or @method('DELETE') in your Blade forms to fake these methods, which is why you see @method('DELETE') in your delete form and @method('PUT') in your edit form.

6. The Controller — PostController.php

The controller is where all the logic of your app lives. It is located at:

```
app/Http/Controllers/PostController.php
```

Think of each method in the controller as handling one specific action the user can take. Let's go through each one.

index() — Show All Posts

```
public function index()
{
    $posts = Post::latest()->get();
    return view('posts.index', compact('posts'));
}
```

This method runs when you visit /posts. It fetches all posts from the database ordered by newest first, then passes them to the index.blade.php view. The compact('posts') function packages the \$posts variable so Blade can use it.

create() — Show the Create Form

```
public function create()
{
    return view('posts.create');
}
```

This just shows the blank create form. No database interaction needed — it simply returns the create.blade.php view.

store() — Save a New Post

```
public function store(Request $request)
{
    $request->validate(['title'=>'required', 'body'=>'required']);
    Post::create($request->all());
    return redirect()->route('posts.index')->with('success', 'Post
created.');
```

This runs when you submit the create form. It first validates that both title and body are not empty. If validation fails, Laravel automatically sends the user back to the form with error messages. If validation passes, it creates a new row in the database, then redirects back to the post list with a success message.

show() — View a Single Post

```
public function show(Post $post)
{
    return view('posts.show', compact('post'));
}
```

Notice that the parameter is Post \$post — not just an ID number. This is called Route Model Binding. Laravel automatically looks up the post by its ID from the URL and injects the Post object directly. You don't need to write Post::find(\$id) yourself.

edit() — Show the Edit Form

```
public function edit(Post $post)
{
    return view('posts.edit', compact('post'));
}
```

Same as show(), this uses Route Model Binding to fetch the post and passes it to the edit form so the existing title and body values are pre-filled in the inputs.

update() — Save Changes to a Post

```
public function update(Request $request, Post $post)
{
    $request->validate(['title'=>'required','body'=>'required']);
    $post->update($request->all());
    return redirect()->route('posts.index')->with('success','Post
updated.');
```

This runs when you submit the edit form. It validates the data, then calls `$post->update()` which updates only the fillable fields in the database. Then it redirects back to the list.

destroy() — Delete a Post

```
public function destroy(Post $post)
{
    $post->delete();
    return redirect()->route('posts.index')->with('success','Post
deleted.');
```

This runs when you click the Delete button. It calls `$post->delete()` which removes the row from the database, then redirects back to the list.

7. Blade Views — The HTML Pages

Blade is Laravel's templating engine. It lets you write HTML with special Laravel syntax mixed in. All Blade files end in `.blade.php` and live inside `resources/views/posts/`.

What Makes Blade Special?

Instead of plain HTML, Blade lets you use PHP logic directly in your HTML using clean syntax:

`{{ $variable }}` — Outputs a variable safely (automatically escapes HTML to prevent attacks)

`@if / @endif` — Conditional logic

`@foreach / @endforeach` — Loop through a list of items

`{{ route('posts.create') }}` — Generates a URL from a route name

`@csrf` — Adds a hidden security token to forms (protects against CSRF attacks)

`@method('DELETE')` — Fakes a DELETE or PUT HTTP method inside an HTML form

index.blade.php — The Post List

This is your main page at `/posts`. It receives the `$posts` variable from the controller and loops through each one using `@foreach` to display the title, body, and action links for every post.

The `session('success')` check shows a green success message after creating, updating, or deleting a post. Laravel stores this message temporarily in the session so it survives the redirect.

create.blade.php — The Create Form

A simple HTML form that sends a POST request to `/posts` when submitted. The `@csrf` directive adds a hidden security token that Laravel checks on every form submission to make sure the request is legitimate.

edit.blade.php — The Edit Form

Very similar to `create.blade.php`, but the input fields are pre-filled with the existing post data using `value="{{ $post->title }}"` and `{{ $post->body }}`. It also uses `@method('PUT')` because editing uses a PUT request, not POST.

show.blade.php — View a Single Post

The simplest view — just displays the title and body of a single post, with a back link to return to the list.

8. The Full Request Flow (Step by Step)

Here is exactly what happens when you visit `http://127.0.0.1:8000/posts` — the complete journey from browser to screen:

1. You type the URL in your browser and press Enter.
2. PHP's built-in server (started by `php artisan serve`) receives the request.
3. Laravel checks `routes/web.php` and matches `/posts` to the `index()` method of `PostController`.
4. `PostController@index` runs: `Post::latest()->get()` sends a `SELECT` query to MySQL and returns all posts as PHP objects.
5. The controller passes the `$posts` collection to the view: `return view('posts.index', compact('posts'))`.
6. Laravel compiles `index.blade.php` — it processes all `@foreach`, `{{ }}` and other Blade syntax — into plain HTML.
7. That HTML is sent back to your browser and displayed on screen.

The whole process: This entire journey takes milliseconds. Every single page load in your app follows this same pattern: Request → Routes → Controller → Model → View → Response.

9. Artisan — Laravel's Command Line Tool

Artisan is Laravel's built-in command-line interface. You used it several times already. Here is what each command does:

`php artisan serve` — Starts a local development server at `http://127.0.0.1:8000`

`php artisan make:model Post -m` — Creates `Post.php` model AND a migration file at the same time (`-m` flag)

`php artisan make:controller PostController --resource` — Creates a controller with all 7 CRUD methods pre-written (`--resource` flag)

`php artisan migrate` — Runs all pending migrations and creates/updates database tables

Artisan has dozens of other commands you'll discover as you go further with Laravel. You can see all available commands by running:

```
php artisan list
```

10. CRUD Summary

CRUD stands for Create, Read, Update, Delete — the four basic operations of any data-driven app. Here is how each one maps to your blog app:

CREATE: User fills out create.blade.php form → POST /posts → store() method → Post::create() → new row in database

READ: User visits /posts → index() method → Post::latest()->get() → index.blade.php displays all posts. Or visits /posts/1 → show() method → single post displayed

UPDATE: User clicks Edit → edit.blade.php form pre-filled → submits → PUT /posts/1 → update() method → \$post->update() → row updated in database

DELETE: User clicks Delete button → DELETE /posts/1 → destroy() method → \$post->delete() → row removed from database

11. Quick Reference — Files You Touch

.env — Database connection settings

database/migrations/xxxx_create_posts_table.php — Defines the posts table columns

app/Models/Post.php — Represents a post, defines \$fillable

app/Http/Controllers/PostController.php — All CRUD logic (7 methods)

routes/web.php — Defines all URL routes

resources/views/posts/index.blade.php — List all posts page

resources/views/posts/create.blade.php — Create new post form

resources/views/posts/edit.blade.php — Edit post form

resources/views/posts/show.blade.php — View single post page

public/css/ — Your CSS stylesheets

You Did It!

Even though it felt overwhelming at first, you have actually built a fully functional web application that connects to a real database, handles multiple routes, validates user input, and renders dynamic HTML. That is not a small thing.

The structure of your blog app — Model, View, Controller, Routes, Migrations — is the exact same structure used in real Laravel applications. Once you understand it here, you understand it everywhere.

Next steps to explore: Blade layouts (so you don't repeat HTML on every page), form validation error messages, pagination, user authentication with `php artisan make:auth` or Laravel Breeze.